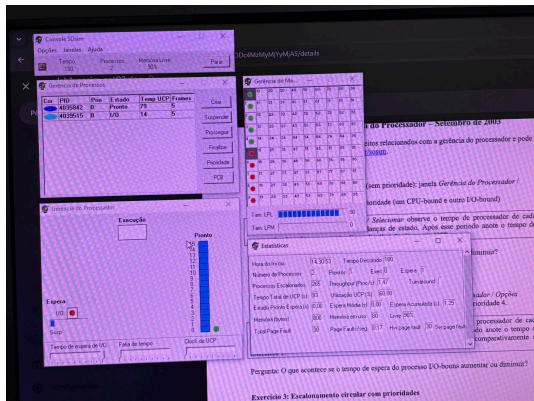


Exercício 1 — Escalonamento circular

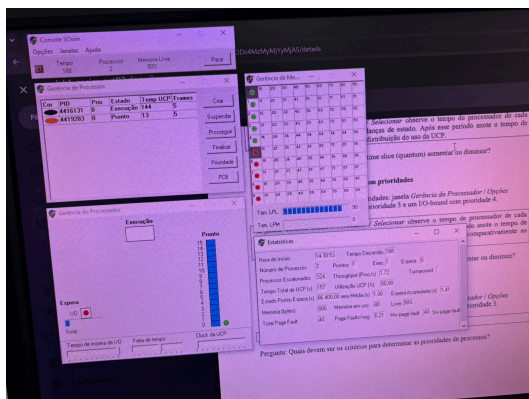


Tempos observados:

- CPU-bound: **79 s**
- I/O-bound: **14 s**

Durante a execução, o processo CPU-bound usou a CPU por bem mais tempo que o I/O-bound. Isso acontece porque o CPU-bound passa a maior parte do tempo pronto para executar, enquanto o I/O-bound precisa parar várias vezes para esperar operações de entrada e saída.

Pergunta: O que acontece se o tempo de time slice (quantum) aumentar ou diminuir?



Quando alteramos o quantum, também muda a frequência com que os processos se alternam na CPU. Com um quantum maior, cada processo pode ficar mais tempo executando antes de dar lugar ao próximo. Isso reduz a quantidade de trocas de contexto, mas pode fazer os outros processos esperarem mais.

Já com um quantum menor, as trocas acontecem com mais frequência. Isso pode melhorar o tempo de resposta dos processos, mas também aumenta o número de trocas de contexto.

Na segunda execução, observamos **142 s para o CPU-bound e 13 s para o I/O-bound**, mostrando que a mudança no quantum também afetou a forma como o tempo da CPU foi distribuído.

Exercício 2 — Escalonamento circular com prioridades

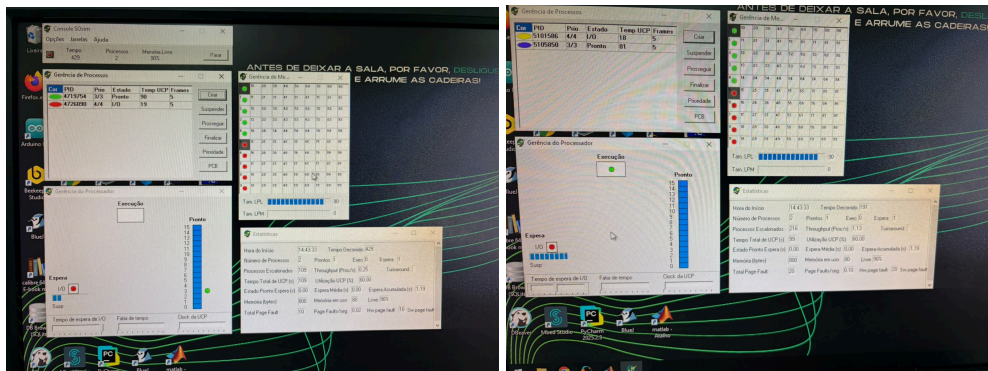
Com tempo de espera de I/O maior:

- CPU-bound: **90 s**
- I/O-bound: **19 s**

Com tempo de espera de I/O menor:

- CPU-bound: **81 s**
- I/O-bound: **18 s**

Pergunta: O que acontece se o tempo de espera do processo I/O-bound aumentar ou diminuir?

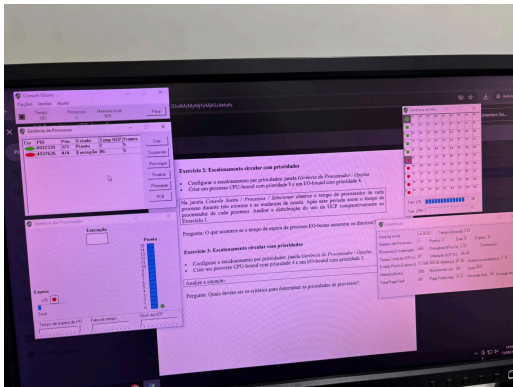


Quando o tempo de espera aumenta, o processo I/O-bound passa mais tempo bloqueado esperando a operação de entrada e saída terminar. Enquanto isso, o CPU-bound fica mais disponível para usar o processador.

Quando diminuirmos esse tempo, o I/O-bound consegue voltar mais rápido para o estado de pronto e disputar novamente a CPU.

Nos nossos testes, os valores ficaram relativamente próximos. Com o tempo de espera maior, tivemos **90 s para o CPU-bound e 19 s para o I/O-bound**. Com a espera menor, tivemos **81 s e 18 s**, respectivamente. Mesmo assim, nos dois casos o CPU-bound continuou utilizando a CPU por mais tempo.

Exercício 3 — Escalonamento circular com prioridades



Tempos observados:

- CPU-bound, prioridade 3: **2 s**
- I/O-bound, prioridade 4: **85 s**

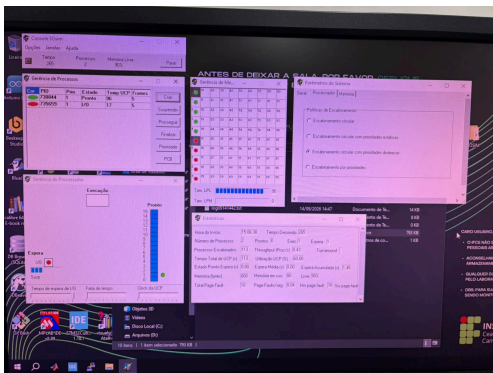
Nesse teste, a diferença ficou bem mais evidente. O processo I/O-bound, que estava com prioridade 4, acumulou **85 s de CPU**, enquanto o CPU-bound, com prioridade 3, ficou com apenas **2 s**.

Pergunta: Quais devem ser os critérios para determinar as prioridades de processos?

A prioridade deve levar em consideração o comportamento e a necessidade de cada processo. Alguns pontos importantes são o quanto ele utiliza a CPU, a quantidade de operações de entrada e saída, a necessidade de responder rapidamente e a importância daquela tarefa para o sistema.

Também é importante evitar que um processo de prioridade mais baixa fique esperando por tempo demais sem conseguir executar. O nosso experimento mostra bem como a prioridade pode fazer bastante diferença na distribuição do processador.

Exercício 4 — Escalonamento circular com prioridades dinâmicas



Tempos observados:

- CPU-bound: **96 s**
- I/O-bound: **17 s**
- Ambos iniciaram com **prioridade 1**.

Nesse teste usamos o escalonamento circular com **prioridades dinâmicas**. Diferente do Exercício 2, em que definimos prioridades diferentes manualmente para os processos, aqui os dois começaram com a mesma prioridade e o próprio sistema pode ajustá-la de acordo com o comportamento de cada processo.

Comparando os resultados, no Exercício 2 obtivemos **90 s para o CPU-bound e 19 s para o I/O-bound** no teste com maior espera de I/O, enquanto neste exercício obtivemos **96 s e 17 s**, respectivamente. Os tempos ficaram próximos, mas a diferença principal está na forma como as prioridades são definidas: no Exercício 2 elas são **estáticas**, enquanto no Exercício 4 são **dinâmicas e adaptadas durante a execução**.

Pergunta: Qual a vantagem desse escalonamento em processos I/O-bound de perfis diferentes?

A vantagem é que não precisamos definir uma prioridade fixa que funcione bem para todos os processos I/O-bound. Como eles podem ter tempos e frequências de espera de I/O diferentes, o escalonamento dinâmico consegue adaptar as prioridades durante a execução. Assim, quando um processo termina sua espera de I/O e volta a ficar pronto, ele pode ser favorecido para voltar a executar mais rapidamente, tornando o escalonamento mais adequado ao comportamento real de cada processo.